

# STATISTICAL ANALYSIS WEB APP — SHINYLIVE PROJECT (REVISED PLAN)

## PROJECT OVERVIEW

Build a Shinylive (WebR) web application that enables researchers to upload data files and receive publication-ready statistical tables and plots. All analysis runs entirely in the user's browser — data never leaves their computer. Includes optional causal inference guidance with DAG visualization.

---

## PHASE 0: WEBR PACKAGE VALIDATION (Before Any Application Code)

**Duration: 1–2 days. This phase gates the entire project.**

### 0.1 Create Minimal Test App

Build a single-file Shinylive app ( `test_app.R` ) that attempts to load every planned package and run one trivial operation with each:

```
# Phase 0 test app structure
library(shiny)

# Tier 1: Core (must pass)
library(gt)
library(gtsummary)
library(ggplot2)
library(broom)
library(labelled)

# Tier 2: Analysis (must pass for MVP)
library(survival)
library(sandwich)
library(lmtest)
library(car)
library(emmeans)

# Tier 3: File I/O (test, have fallbacks)
library(haven)
```

```
library(readxl)

# Tier 4: Advanced (nice to have)
library(lme4)
library(ggdag)
library(writexl)
```

## 0.2 Functional Smoke Tests

For each loaded package, run one representative operation:

| Package   | Test                                                            |
|-----------|-----------------------------------------------------------------|
| gt        | <code>gt(mtcars[1:5,]) %&gt;% as_raw_html()</code>              |
| gtsummary | <code>tbl_summary(mtcars[,1:4])</code>                          |
| ggplot2   | <code>ggplot(mtcars, aes(mpg, hp)) + geom_point()</code>        |
| broom     | <code>tidy(lm(mpg ~ hp, mtcars))</code>                         |
| survival  | <code>survfit(Surv(time, status) ~ x, aml)</code>               |
| lme4      | <code>`lmer(Reaction ~ Days + (1</code>                         |
| haven     | <code>read_dta()</code> on a small bundled test file            |
| readxl    | <code>read_xlsx()</code> on a small bundled test file           |
| ggdag     | <code>dagify(y ~ x) %&gt;% ggdag()</code>                       |
| writexl   | <code>write_xlsx(mtcars, "/tmp/test.xlsx")</code> then download |
| sandwich  | <code>vcovCL(lm(mpg~hp, mtcars), cluster=mtcars\$cyl)</code>    |

## 0.3 Performance Benchmarks

With a ~1 MB CSV dataset (n ≈ 5,000 rows, 20 columns):

- Time to upload and parse
- Time for descriptive statistics (Table 1)
- Time for descriptive statistics with 30 variables (stress test for gtsummary)
- Time for descriptive statistics with a high-cardinality categorical (20+ levels)

- Time for linear regression with 5 predictors
- Time for logistic regression with 5 predictors
- Time for survival analysis (Cox model)
- Time for mixed model (if lme4 loads)
- Memory usage observed in browser DevTools (Chrome and Safari)
- Safari-specific memory test: repeat full workflow twice without page reload

## 0.4 Decision Matrix

After testing, classify each package:

| Result                              | Action                                             |
|-------------------------------------|----------------------------------------------------|
| Loads and works                     | Include in v1                                      |
| Loads but slow (>10 s on benchmark) | Include with performance warning, cap dataset size |
| Loads but partial functionality     | Include with documented limitations                |
| Fails to load                       | Use fallback (see Fallback Table below)            |

## 0.5 Fallback Table

| Primary Package | Fallback                          | What Changes                                             |
|-----------------|-----------------------------------|----------------------------------------------------------|
| haven           | CSV-only input for v1             | Remove Stata/SPSS from v1 file support                   |
| readxl          | CSV-only input for v1             | Remove Excel from v1 file support                        |
| lme4            | Drop mixed models from v1         | Treat as experimental even if Phase 0 passes (see below) |
| ggdag / dagitty | Pure ggplot2 manual DAG rendering | Simpler static DAG, no interactive editing               |
| writexl         | CSV export + HTML tables          | Users get CSV data + formatted HTML                      |
| officer         | Not used (see Export Strategy)    | —                                                        |
|                 |                                   |                                                          |

|                 |                                        |                                                                                  |
|-----------------|----------------------------------------|----------------------------------------------------------------------------------|
| flextable       | Not used; gt is primary                | —                                                                                |
| openxlsx        | Not used; writexl is primary           | —                                                                                |
| car             | Manual implementation of VIF, Levene's | Implement key diagnostics in base R                                              |
| sandwich / lme4 | Disable cluster-robust SEs for v1      | Note clustering in output but do not adjust SEs; offer mixed model if lme4 works |

**lme4 policy:** Even if lme4 loads and passes smoke tests, treat it as *experimental* in v1. Label the UI clearly: "Experimental: may be slow or unstable in browser." Do not promise mixed models as a core feature. If lme4 is unusably slow on the benchmark dataset (>30 s for a simple random-intercept model), disable it entirely.

**sandwich policy:** Cluster-robust SEs are easy to get subtly wrong in a manual implementation. If `sandwich` fails Phase 0, do *not* implement a manual variance estimator. Instead, note clustering in the output and offer the mixed model path (if lme4 is available) or advise the user to run locally. This is safer than shipping a potentially incorrect estimator that users will trust without checking.

## 0.6 Phase 0 Deliverable

A written log documenting: which packages loaded, which failed, benchmark timings, and the confirmed package list for v1. **Do not proceed to Phase 1 until this is complete.**

The Phase 0 deliverable must include:

1. **Package load results table:** Every package listed with status (✅ loaded / ❌ failed / ⚠️ partial), error messages if any, and load time
2. **Functional test results:** Pass/fail for each smoke test in §0.2, with error output for failures
3. **Benchmark timings table:** Every benchmark in §0.3 with wall-clock time in seconds and browser used
4. **Safari memory test result:** Did the repeated workflow complete without crash or excessive slowdown?
5. **Written decision log:** For each package, the decision (include / exclude / experimental) with rationale, committed to the repo as `PHASE0_VALIDATION.md`
6. **Confirmed v1 package list:** The final set of packages that will ship

This is a hard gate. No application code should be written until this document exists and all decisions are recorded.

---

## TECHNICAL STACK (Revised)

- **Framework:** Shinylive (Shiny + WebR)
- **Primary table package:** `gt` (confirmed WebR-compatible; replaces `flextable/kableExtra`)
- **Primary summary package:** `gtsummary` (confirmed WebR-compatible)
- **Plotting:** `ggplot2`
- **Model tidying:** `broom`
- **Variable labels:** `labelled`
- **File reading (if Phase 0 passes):** `haven` , `readxl` ; otherwise CSV only
- **File writing:** `writexl` (Excel export), `gt` (HTML export)
- **Statistical modelling:** base R ( `lm` , `glm` , `t.test` , `chisq.test` , `wilcox.test` , `fisher.test` , `aov` ), `survival` , `emmeans`
- **Cluster-robust SEs:** `sandwich` , `lmtest` (if Phase 0 passes); otherwise manual calculation
- **Mixed models (if Phase 0 passes):** `lme4`
- **DAG visualization (if Phase 0 passes):** `ggdag` (wraps `dagitty` )
- **Diagnostics:** `car` (if available); otherwise base R implementations

### Packages Removed from Plan

| Removed                 | Reason                                                 | Replacement                   |
|-------------------------|--------------------------------------------------------|-------------------------------|
| <code>flextable</code>  | gdtools/curl dependency risk in WASM                   | <code>gt</code>               |
| <code>kableExtra</code> | Redundant with <code>gt</code> ; uncertain WASM status | <code>gt</code>               |
| <code>officer</code>    | System library dependencies; Word                      | HTML + copy-to-Word workflow; |

|          |                                    |                          |
|----------|------------------------------------|--------------------------|
|          | export unreliable in WASM          | JS-based docx generation |
| openxlsx | Unmaintained; Rcpp dependency risk | writexl                  |

## EXPORT STRATEGY (Revised — Critical Section)

The original plan relied on `officer` for Word and `openxlsx` for Excel. Both are high-risk in WebR. The revised strategy uses browser-native approaches.

### Tables: Copy-to-Word Workflow (Primary)

1. **Render tables as styled HTML** using `gt::as_raw_html()` within the Shiny app
2. User clicks **"Copy Table"** button → JavaScript copies the HTML table to clipboard
3. User pastes into Word → Word preserves formatting (bold headers, borders, alignment)
4. This works reliably because Word interprets pasted HTML tables natively

Implementation:

```
# Server-side: render gt table as HTML
output$table_html <- render_gt({ ... })

# UI: Copy button using JavaScript clipboard API
actionButton("copy_table", "Copy Table to Clipboard", icon = icon("clipboard"))

# JavaScript handler (in www/clipboard.js or inline)
# Uses navigator.clipboard.write() with text/html MIME type
```

### Tables: Download Options

| Format                   | Method                                                                   | Reliability in WebR         |
|--------------------------|--------------------------------------------------------------------------|-----------------------------|
| <b>HTML<br/>(.html)</b>  | <code>gt::gtsave()</code> to temp file →<br><code>downloadHandler</code> | ✅ High — pure text output   |
| <b>CSV (.csv)</b>        | <code>write.csv()</code> → <code>downloadHandler</code>                  | ✅ High — always works       |
| <b>Excel<br/>(.xlsx)</b> | <code>writexl::write_xlsx()</code> →<br><code>downloadHandler</code>     | ⚠️ Medium — test in Phase 0 |
|                          |                                                                          |                             |

|                         |                                                            |                                |
|-------------------------|------------------------------------------------------------|--------------------------------|
| <b>LaTeX<br/>(.tex)</b> | <code>gt::as_latex()</code> → <code>downloadHandler</code> | ✅ High — pure text output      |
| <b>Word<br/>(.docx)</b> | JavaScript-based generation (see below)                    | ⚠️ Medium — requires JS bridge |

## Word Document Export (JavaScript Approach)

Since `officer` is unreliable in WASM, generate `.docx` files client-side using JavaScript:

1. Include `docx.js` library in the Shinylive app's `www/` folder
2. R generates the data (table contents, statistics, plot file paths)
3. Pass data to JavaScript via `session$sendCustomMessage()`
4. JavaScript builds the `.docx` using `docx.js` and triggers download via `Blob + URL.createObjectURL()`

This runs entirely in the browser and avoids all R-side Word generation packages.

Alternatively, for a simpler v1: provide HTML download and a note “Open in Word: File → Open → select the `.html` file” — Word converts HTML tables cleanly.

## Plots: Download Options

| Format                       | Method                                                                              | Reliability in WebR                                |
|------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------|
| <b>PNG<br/>(300<br/>DPI)</b> | <code>ggsave()</code> with <code>png()</code> device → <code>downloadHandler</code> | ✅ High — built-in graphics device                  |
| <b>SVG</b>                   | <code>ggsave()</code> with <code>svg()</code> device → <code>downloadHandler</code> | ✅ High — built-in                                  |
| <b>PDF</b>                   | <code>ggsave()</code> with <code>pdf()</code> device → <code>downloadHandler</code> | ⚠️ Medium — depends on Cairo support in WebR build |

## Combined Report

For v1, generate a single **HTML report** containing all selected tables and plots:

- Use `htmltools` to compose the page (confirmed working in WebR)
- Include inline CSS for professional styling (Default or APA formatting)

- Tables rendered as styled HTML
- Plots embedded as base64-encoded PNG images
- User downloads as `.html` file
- Can be opened directly in Word (File → Open) with good fidelity

For v2, add JavaScript-based `.docx` report generation.

---

## CORE FEATURES

### 1. DATA UPLOAD & PROCESSING

#### Supported formats (confirmed by Phase 0):

- CSV (always available — base R)
- Excel (`.xlsx`, `.xls`) — via `readxl` if Phase 0 passes
- Stata (`.dta`) — via `haven` if Phase 0 passes
- SPSS (`.sav`) — via `haven` if Phase 0 passes
- R (`.rds`) — via base R `readRDS()`, always available

**If `haven` or `readxl` fail Phase 0:** The app supports CSV and `.rds` only in v1, with a message: "Stata, SPSS, and Excel support coming soon. Please convert your data to CSV."

**File size limit:** 5 MB for v1 (reduced from 10 MB based on WASM performance concerns). Display a clear message if exceeded. Reassess after performance benchmarking.

**Dataset dimension guard:** After parsing, check  $n \times p$  (rows  $\times$  columns). If  $n \times p > 500,000$ , display warning: "This dataset may be too large for reliable in-browser analysis. Consider reducing variables or rows, or running locally in R." If  $n \times p > 1,000,000$ , hard stop with message: "This dataset exceeds the browser's capacity. Please run this analysis locally in R or reduce the dataset size." These thresholds should be calibrated during Phase 0 benchmarking — Safari (especially iPad) has significantly lower memory limits than Chrome.

**Browser memory monitoring:** Where possible, use `performance.memory` (Chrome) or similar APIs to detect memory pressure. If memory usage exceeds 80% of available heap, display warning and suggest reducing dataset size.

#### Variable labels:

- Auto-detect labels from Stata, SPSS, R labeled data (via `labelled` package)

- Extract and preserve for use in all outputs
- If no label exists, use variable name
- User can edit labels in Step 1

### Variable type detection:

- Auto-detect: binary, categorical, continuous, identifier
- Logic: unique values  $\leq 2 \rightarrow$  binary; unique values 3–10  $\rightarrow$  categorical; unique values  $>10$  and numeric  $\rightarrow$  continuous; character with many unique values  $\rightarrow$  identifier
- Allow user override with dropdown selectors
- **Performance guard:** If a categorical variable has  $>20$  levels, auto-suggest collapsing rare categories into "Other ( $<5\%$ )" or reclassifying as continuous/identifier. Warn: "Variables with many categories slow down Table 1 generation."
- **Table 1 variable cap:** Limit Table 1 to 30 variables maximum. If the dataset has more, user must select which to include. This prevents gtsummary from freezing the browser on wide datasets.

### Clustering detection:

- Auto-detect patterns: repeated IDs, hierarchical naming, panel structure
- Look for common cluster variable names (id, cluster, group, site, school\_id, etc.)
- If detected, display non-blocking alert

### Missing value handling:

- Smart detection: NA, "", blank cells, -999, program-specific conventions
- If unclear, prompt user: "How are missing values coded in your data?"
- User selects treatment: pairwise deletion (default), complete case analysis, simple imputation (mean/mode)
- Display note: "For complex missing data patterns, consider multiple imputation outside this tool"

## 2. STATISTICAL ANALYSES SUPPORTED

### v1 (confirmed packages):

- Descriptive statistics (Table 1 with stratification) — via `gtsummary`
- t-tests: independent samples, paired samples, one-sample — via base R

- Chi-square tests and Fisher's exact test — via base R
- ANOVA: one-way, two-way — via base R `aov()`
- Linear regression (simple and multiple) — via base R `lm()`
- Logistic regression (binary outcomes) — via base R `glm()`
- Survival analysis (Kaplan-Meier, Cox regression) — via `survival`
- Correlation and simple tests (Pearson, Spearman, Wilcoxon, etc.) — via base R

#### **v1 if Phase 0 passes:**

- Mixed effects models (linear and logistic) — via `lme4` — **EXPERIMENTAL: labelled as such in UI, may be disabled if too slow**
- Cluster-robust standard errors — via `sandwich` + `lmtest` — **if `sandwich` unavailable, clustering is noted but SEs are not adjusted**
- Repeated measures ANOVA — via base R or `lme4`

#### **Deferred to v2 if Phase 0 fails:**

- Mixed effects models
- Cluster-robust standard errors (noted but unadjusted in v1)

### **3. USER INTERFACE — MULTI-STEP WIZARD**

**Architecture:** Each wizard step is a Shiny module ( `moduleServer` / `NS` ) with a defined interface. A central `reactiveValues` list holds the full session state.

#### **State management schema:**

```
state <- reactiveValues(
  # Step 1
  raw_data = NULL,
  variable_info = NULL,      # data.frame: name, label, type, n_missing, pct_missin
  user_labels = NULL,        # named list of user-edited labels
  user_types = NULL,         # named list of user type overrides
  missing_strategy = "pairwise",
  cluster_var = NULL,

  # Step 2
  selected_analysis = NULL,

  # Step 3
```

```

exposure_vars = NULL,
outcome_vars = NULL,
covariates = NULL,

# Step 4 (optional)
causal_roles = NULL,      # named list: variable -> role
dag_object = NULL,
include_dag = FALSE,

# Step 5
table_style = "default",
use_clustering = FALSE,
cluster_method = "robust_se",
include_diagnostics = FALSE,
confidence_level = 0.95,

# Step 6
results = NULL,
selected_outputs = NULL
)

```

Include **"Quick Mode"** toggle at top: uploads data (Step 1 still required with defaults auto-accepted), then skips directly to Step 3 (model specification). Quick Mode uses auto-detected variable types, default labels, and pairwise deletion without review.

**Progress indicator:** Upload → Explore → Choose Analysis → Model → [Causal] → Options → Results

## STEP 1: UPLOAD & EXPLORE

- File upload interface with drag-drop
- Display:
  - Variable names and labels (if available)
    - Format: "Label (variable\_name)" or just "variable\_name" if no label
  - Auto-detected types (binary/categorical/continuous/identifier)
  - Sample values (first 5 unique values)
  - Missing data summary (count and % missing per variable)
  - Clustering alert if detected: "Possible clustered data detected (variable: [name]). Your data may have clustering which affects standard errors."

- File format note if CSV: "For richer metadata (variable labels, value labels), consider uploading Stata or SPSS files."
- User actions:
  - "Edit Labels" button → Opens modal with editable text fields per variable
  - Override variable types via dropdown (binary/categorical/continuous/identifier)
  - Specify missing value handling
  - Preview first 50 rows of data
- Performance indicator: if file size > 2 MB, display "Large file — analysis may take longer in browser"
- "Next" button to proceed

## STEP 2: CHOOSE ANALYSIS

- Display available analyses grouped by category:
  - Descriptive Statistics
  - Group Comparisons (t-tests, ANOVA, chi-square)
  - Regression (linear, logistic)
  - Survival Analysis
  - Advanced (mixed models — shown only if `lme4` passed Phase 0)
- Smart suggestions based on variable types:
  - "You have a binary outcome and continuous predictors → Logistic regression recommended"
  - "You have two groups and a continuous outcome → Independent samples t-test recommended"
  - "You have a time-to-event variable → Survival analysis recommended"
- Brief plain-language description of each analysis type
- Allow multi-select for descriptive stats + inferential test
- "Next" button

## STEP 3: SPECIFY MODEL

- **Variable selection interface** using grouped multi-select inputs (dropdown selectors with search):
  - Exposure/Independent Variable(s) selector
  - Outcome/Dependent Variable(s) selector
    - Allows multiple outcomes
    - Badge showing count: "3 outcomes selected"
  - Covariates/Adjustment Variables selector
- Variables displayed as "Label (variable\_name)" in all selectors
- Drag-and-drop interface is a **v2 enhancement** (use the `sortable` package if available in WebR; otherwise standard Shiny select inputs are the reliable default)
- Multiple outcomes section:
  - When >1 outcome selected, show summary:
    - "Analyzing multiple outcomes with the same exposure and covariates"
    - List of selected outcomes with remove buttons
- Validation: check that appropriate variable types are assigned for the selected analysis. Display clear error messages ("Logistic regression requires a binary outcome variable")
- Clear labels and tooltips
- "Help me think through causality" button (launches Step 4)
- "Skip causal reasoning" button (goes to Step 5)

## STEP 4: [OPTIONAL] CAUSAL REASONING MODULE

Only appears if user clicks "Help me think through causality."

- For each covariate, ask:
  - "What is the role of [variable label/name] in your causal model?"
  - Options (radio buttons):
    - Confounder (causes both exposure and outcome)
    - Mediator (on causal pathway between exposure and outcome)
    - Effect modifier (changes strength of exposure-outcome relationship)

- Collider (caused by both exposure and outcome)
  - Precision variable (reduces variance but not a confounder)
- Brief plain-language tooltip for each option
- **Static DAG rendering** (v1): generate a DAG from the user's answers using `ggdag` (if available) or pure `ggplot2` with manual arrows:
  - Exposure → Outcome (main arrow)
  - Covariates with appropriate arrows based on stated roles
  - Clean, readable layout
  - No interactive editing in v1
- **Interactive DAG editor** deferred to v2 (complex JS/Shiny integration)
- Warnings/Alerts displayed prominently:
  - "You're adjusting for a mediator ([variable]) — this may bias your exposure effect estimate. Consider removing it."
  - "You're conditioning on a collider ([variable]) — this may introduce spurious associations. Consider removing it."
- Export DAG: download as PNG (via `ggsave` )
- "Update Model" button → returns to Step 3 with modified covariate selection
- "Continue to Analysis" → proceeds to Step 5

## STEP 5: ANALYSIS OPTIONS

### Table Formatting:

- Default: Clean academic style
- Style selector (dropdown):
  - Default (generic clean)
  - APA (American Psychological Association)
- Additional styles deferred to v2: AMA, Vancouver, JAMA (each adds testing surface with minimal v1 value)

### Clustered/Grouped Data:

- “My data has clustering” checkbox
- If checked:
  - Dropdown: “Select cluster variable:” [list of all variables, pre-selected if auto-detected]
  - Tooltip: “Clustering adjusts standard errors for non-independence (e.g., students within schools, repeated measures on same person)”
  - Method selection (shown based on Phase 0 results):
    - Cluster-robust standard errors (shown only if `sandwich` passed Phase 0)
    - Mixed model with random intercepts (shown only if `lme4` passed Phase 0, labelled “Experimental”)
    - Note clustering only (always available — adds footnote to output without adjusting SEs)

**Diagnostics:** checkbox to include diagnostic plots (residuals, Q-Q, influence)

**Analysis-specific options:**

- Confidence level: 90%, 95% (default), 99%
- For survival analysis: time units (days/months/years), censoring variable
- For stratified descriptive stats: grouping variable

**Multiple comparisons:**

- If multiple outcomes selected or >2 groups in ANOVA: show checkbox “Apply multiple comparison correction”
  - Options: Bonferroni, Benjamini-Hochberg (FDR)
  - Default: display note about multiple testing even if correction not applied

**Assumptions check (automatic):**

- Before running analysis, briefly check:
  - Sample size adequacy per group (warn if any group <20)
  - Group balance (warn if groups differ >5:1 in size)
  - For regression: check for perfect collinearity
  - Display plain-language warnings: “Your smallest group has only 8 observations —

results may be unreliable"

- These are informational only; user can proceed regardless

"Run Analysis" button (large, prominent)

## STEP 6: RESULTS

### Tables section:

#### *Table 1: Descriptive Statistics*

- Title: "Table 1. Descriptive Statistics (n = [overall n])"
- Column structure:
  - Unstratified: Variable | n | Overall (n=X)
  - Stratified: Variable | n | Overall (n=X) | Group 1 (n=Y) | Group 2 (n=Z) | p-value
- Row specifications:
  - Continuous: display based on distribution (mean  $\pm$  SD if approximately normal, median [IQR] if skewed; use skewness > |1| rather than Shapiro-Wilk to decide)
  - Categorical/Binary: n (%)
  - Variable labels used throughout: "Label (variable\_name)"
- Footnotes: tests used, data presentation format, clustering note if applicable
- Generated via `gtsummary::tbl_summary()` , rendered via `gt`

#### *Analysis Results Tables:*

- For single outcome: regression coefficients with CI and p-values, model fit statistics
- For multiple outcomes:
  - Default: separate tables per outcome
  - Option: combined table with outcomes in columns
- All tables use variable labels
- Table style applied per Step 5 selection (Default or APA)
- If clustering: noted in footer

### Plots section (auto-generated):

All plots use variable labels for axes, legends, and captions. All use `theme_classic()` or `theme_bw()` with 12pt axis labels, 10pt tick labels, colorblind-friendly palette (viridis).

| Analysis                   | Plots Generated                                                                     |
|----------------------------|-------------------------------------------------------------------------------------|
| Linear/Logistic regression | Forest plot (coefficients + 95% CI), effect plot (predicted values across IV range) |
| t-tests/ANOVA              | Box plots or violin plots by group, dot plot with means and error bars              |
| Survival analysis          | Kaplan-Meier curves (stratified if applicable), forest plot for hazard ratios       |
| Descriptive statistics     | Histograms (continuous), bar charts (categorical)                                   |
| Multiple outcomes          | Multi-outcome forest plot comparing exposure effect across outcomes                 |
| Diagnostics (if selected)  | Residual plots, Q-Q normality plots, Cook's distance                                |

### Refinement interface:

After results are displayed:

- Tables: checkboxes to include/exclude each table, toggle min/max values, toggle test statistics, select/deselect individual variables from Table 1
- Plots: checkboxes to include/exclude each plot from download
- For multiple outcomes: per-outcome checkboxes
- Real-time preview updates as checkboxes change

### Download section (revised):

Prominent download area with clearly grouped options:

*Quick Copy (one click):*

- **"Copy Table to Clipboard"** button for each table → Copies styled HTML; pastes directly into Word/Google Docs with formatting preserved
- **"Copy All Tables"** button → Copies all selected tables sequentially

*Table Downloads:*

- HTML (.html) — opens cleanly in Word via File → Open 

- CSV (.csv) — raw data, always works ✓
- Excel (.xlsx) — via `writexl` if available ⚠
- LaTeX (.tex) — for journal submissions ✓

#### *Plot Downloads:*

- PNG (300 DPI) — publication quality ✓
- SVG — vector, editable ✓
- PDF — vector, if supported ⚠

#### *Combined Report:*

- **HTML Report** (.html) — all selected tables + embedded plots in a single styled document. Can be opened in Word. Includes table of contents, page breaks between sections, professional CSS styling. ✓
- **Word Report** (.docx) — v2 feature (JavaScript-based generation). If implemented early, uses `docx.js` client-side.
- Include DAG (if created in Step 4): checkbox
- Include analysis code (R script): checkbox — generates a reproducible R script (not ShinyLive-specific) that the user can run in standard R

“Download Selected Outputs” button (large, prominent)

“Start New Analysis” button (clears session, returns to Step 1)

## DETAILED SPECIFICATIONS

### VARIABLE LABEL HANDLING

Priority order:

1. User-edited labels (from Step 1)
2. Labels from original file (Stata, SPSS, R)
3. Variable names (if no label)

Display format:

- Tables: “Label (variable\_name)”

- Plots: Label only (variable name in caption if needed)
- Table footnotes: "Variable names shown in parentheses"

Implementation:

- Store labels in `state$user_labels` (reactive)
- Apply to `gtsummary` via `label` argument
- Apply to `ggplot2` via `labs()`
- Preserve through all transformations

## CLUSTERED DATA HANDLING

Auto-detection logic:

- Variables with many repeated values (ratio of unique values to rows < 0.1) that are not the outcome
- Naming patterns: `*_id`, `*ID`, `cluster*`, `group*`, `site*`, `school*`
- Hierarchical relationships detected

If clustering selected:

For regression (if `sandwich` available): use `sandwich::vcovCL()` for cluster-robust SEs.

Display in results: "Standard errors clustered by [label]."

For regression (if `sandwich` unavailable): do **not** attempt manual robust SE calculation.

Instead, display in results: "Clustering by [label] detected. Standard errors are not adjusted for clustering in this version. For cluster-adjusted inference, consider using R locally with the `sandwich` package, or use the mixed model option below." If `lme4` is available, offer: "Alternatively, fit a mixed model with random intercepts for [label]."

For other analyses: note clustering in output; adjust tests if possible (e.g., paired t-test for repeated measures).

## PLOT SPECIFICATIONS

All plots publication-ready:

- Resolution: 300 DPI for PNG
- Vector: SVG always available, PDF if supported
- Palette: viridis or ColorBrewer (colorblind-friendly + grayscale-friendly)

- Fonts: 12pt axis labels, 10pt tick labels (system default in WebR)
- Elements: clear axis labels with variable labels, 95% CI where relevant, appropriately positioned legend, minimal gridlines, white background
- Theme: `theme_classic()` or `theme_bw()`

## TABLE 1 SPECIFICATIONS

- Title: "Table 1. Descriptive Statistics (n = [overall n after exclusions])"
- Distribution assessment: skewness > |1| triggers median [IQR]; otherwise mean  $\pm$  SD (avoids Shapiro-Wilk problems at large n)
- Missing data: if >5% missing for any variable, add footnote; option to add "Missing" row
- Post-generation refinement: checkboxes to remove min/max, remove test statistics, remove specific variables, change stratification variable, show/hide missing counts

## MULTIPLE COMPARISONS

- If >1 outcome analysed: display note "Multiple outcomes tested — consider adjusting for multiple comparisons"
- If >2 groups in ANOVA: offer Bonferroni or Tukey post-hoc tests
- Option to display both unadjusted and adjusted p-values

## ASSUMPTIONS CHECK

Before running inferential analyses, automatically check and report (non-blocking):

| Check                | Trigger                        | Message                                                                                         |
|----------------------|--------------------------------|-------------------------------------------------------------------------------------------------|
| Small sample         | Any group n < 20               | "Your smallest group has [n] observations — results may be unreliable"                          |
| Imbalanced groups    | Group ratio > 5:1              | "Your groups are very unbalanced ([n1] vs [n2]) — consider whether this affects interpretation" |
| Zero variance        | Any predictor has no variation | "[Variable] has no variation and will be dropped from the model"                                |
| Perfect collinearity | VIF > 10 detected              | "High collinearity detected between [var1] and [var2] — consider removing one"                  |
|                      | Model fails to                 | "Model did not converge — try simplifying (fewer                                                |

|             |          |                                |
|-------------|----------|--------------------------------|
| Convergence | converge | predictors or random effects)" |
|-------------|----------|--------------------------------|

---

## SECURITY & PRIVACY

### Landing page banner:



Your data never leaves your computer — all analysis runs in your browser. No servers, no uploads, completely private.

Brief explanation of WebR technology with link to FAQ.

**Upload page reminder:** "Data is processed locally on your device"

### FAQ section:

- "How is my data kept private?" → Detailed explanation of client-side processing, no data transmission, no cloud storage
- "Is this equivalent to running R on my computer?" → Yes, WebR runs R compiled to WebAssembly directly in your browser

### Security measures:

- No telemetry or tracking code
- No external API calls except initial WebR/package download from CDN
- No cookies except for caching WebR runtime locally
- Open source on GitHub for transparency
- Network tab audit: verify NO requests after initial load (documented in testing)

---

## STATE MANAGEMENT

The multi-step wizard uses a single `reactiveValues` object (schema shown in Step 3 above) that persists across all steps. Key design decisions:

- **Back navigation:** All steps read from state; going back and changing something (e.g., variable types in Step 1) propagates automatically to later steps via reactivity
- **Quick Mode:** Sets `state$quick_mode <- TRUE`; Step 1 auto-accepts all defaults; wizard jumps to Step 3

- **Optional Step 4:** Skipping it leaves `state$causal_roles` as NULL; Step 5 and 6 handle this gracefully
  - **Error recovery:** If an analysis fails in Step 6, display the error for that specific analysis/outcome while showing results that succeeded. User stays on Step 6.
- 

## SHINY MODULE STRUCTURE

```

app.R                                     # Entry point, module orchestration
├── R/
│   ├── mod_upload.R                     # Step 1: Upload & Explore
│   ├── mod_choose_analysis.R           # Step 2: Choose Analysis
│   ├── mod_specify_model.R             # Step 3: Model Specification
│   ├── mod_causal.R                    # Step 4: Causal Reasoning (optional)
│   ├── mod_options.R                   # Step 5: Analysis Options
│   ├── mod_results.R                   # Step 6: Results & Download
│   ├── mod_wizard_nav.R                # Progress bar + navigation
│   ├── helpers_analysis.R              # Statistical analysis functions
│   ├── helpers_tables.R                # Table formatting functions
│   ├── helpers_plots.R                 # Plot generation functions
│   ├── helpers_export.R                # Export/download functions
│   ├── helpers_labels.R                # Variable label management
│   └── helpers_validation.R            # Input validation + assumptions checks
├── www/
│   ├── clipboard.js                   # Copy-to-clipboard functionality
│   ├── styles.css                     # Custom styling
│   └── docx_export.js                  # Word export via docx.js (v2)
└── tests/
    ├── test_analysis.R                 # Unit tests for statistical functions
    ├── test_labels.R                   # Unit tests for label handling
    └── test_export.R                   # Unit tests for export functions

```

Each module follows the pattern:

```

mod_upload_ui <- function(id) {
  ns <- NS(id)
  # UI elements
}

mod_upload_server <- function(id, state) {
  moduleServer(id, function(input, output, session) {
    # Logic; reads/writes to shared state
  })
}

```

}

---

## TESTING REQUIREMENTS

### Phase 0 Tests (WebR Validation — See Above)

Performed before any application code. Results determine v1 scope.

### Automated Tests (During Development)

Unit tests using `testthat` (if available in WebR) or manual validation scripts:

#### Statistical correctness:

- Linear regression produces correct coefficients (compare to known result)
- Logistic regression odds ratios match manual calculation
- t-test p-values match base R `t.test()` output
- Table 1 counts are correct (verify against `nrow` and `table` )

#### Variable labels:

- Labels extracted from Stata file match expected values
- User-edited labels persist through analysis
- Labels appear in table output and plot axes

#### Multiple outcomes:

- All outcomes analysed with same model specification
- Separate tables generated for each
- Combined table has correct column structure

#### Edge cases:

- All-missing variable: excluded with informative message
- Perfect collinearity: model still runs with dropped variable + warning
- Zero-variance variable: dropped with warning
- Very small sample ( $n < 10$ ): warning displayed, analysis proceeds

- Single-level categorical variable: detected and flagged
- Large file (approaching 5 MB): completes without browser freeze

#### **Export tests:**

- Copy-to-clipboard produces valid HTML
- HTML download opens in browser
- CSV download contains correct data
- Excel download (if available) opens in Excel
- LaTeX output compiles without errors
- HTML report contains all selected elements

### **Manual Testing (User Performs)**

#### **Statistical validation:**

- Compare results to desktop R/Stata/SPSS for typical analyses
- Have a statistician review outputs for correctness
- Test with real anonymised datasets

#### **Browser compatibility:**

- Chrome (desktop) — full workflow
- Firefox (desktop) — full workflow
- Safari (Mac/iPad) — full workflow
- Edge (desktop) — full workflow
- Mobile: test if usable (expected: functional but not optimal)

#### **Performance:**

- 500 KB file: should be fast (<5 s for analysis)
- 2 MB file: should work smoothly (<15 s for analysis)
- 5 MB file: should complete (may be slow, <60 s)
- First visit load time: target <60 s (monitor this)
- Return visit load time: target <10 s (cached WebR)

### **User experience:**

- Can a colleague with no stats background complete an analysis?
- Are error messages helpful?
- Is the workflow intuitive?
- Can experts use Quick Mode successfully?
- Is the DAG feature useful or confusing?

### **Copy-to-Word workflow:**

- Copy Table 1 → paste into Word → formatting preserved?
- Copy regression table → paste into Word → formatting preserved?
- Test in Word for Mac, Word for Windows, Google Docs, LibreOffice Writer
- Does the HTML report open cleanly in Word via File → Open?

### **Security/Privacy:**

- Open DevTools → Network tab → upload data → verify NO requests except cached WebR
- Check localStorage/sessionStorage — no sensitive data
- Works offline after first load? (Desirable but not required)

---

## **IMPLEMENTATION PHASES (Revised)**

### **Phase 1: Foundation + Wizard Shell (Core Functionality)**

1. Create wizard shell with navigation (all 6 steps as stub modules)
2. Implement shared state management ( `reactiveValues` )
3. File upload and CSV parsing (guaranteed to work)
4. Variable type detection and label handling
5. Basic descriptive statistics via `gtsummary::tbl_summary()`
6. Table 1 generation with `gt` rendering
7. Copy-to-clipboard for tables (JavaScript integration)

8. HTML table download via `downloadHandler`
9. One basic plot (histogram/bar chart for descriptive stats)
10. PNG plot download

**Deliverable:** A working app that takes a CSV, generates Table 1, and lets the user copy it to Word or download it.

## **Phase 2: Statistical Analyses**

11. t-tests (independent, paired, one-sample)
12. Chi-square and Fisher's exact
13. One-way ANOVA
14. Linear regression (simple and multiple)
15. Logistic regression
16. Survival analysis (Kaplan-Meier + Cox)
17. All corresponding auto-generated plots
18. Forest plots for regression coefficients
19. Diagnostic plots (residuals, Q-Q, Cook's distance)
20. Variable labels applied throughout all outputs

**Deliverable:** Full analysis capability with all v1 analyses working end-to-end.

## **Phase 3: Advanced Features**

21. Multiple outcomes support
22. Multi-outcome forest plot
23. Combined table format
24. Causal reasoning module (static DAG, no interactive editing)
25. Clustered data handling (cluster-robust SEs or mixed models)
26. Multiple comparison corrections
27. Assumptions checking
28. Excel and LaTeX export

**Deliverable:** All v1 features complete.

## Phase 4: Polish & Testing

29. Quick Mode implementation
30. Table and plot refinement interface (checkboxes)
31. HTML combined report generation
32. Style options (Default + APA)
33. Comprehensive testing (automated + manual)
34. Performance optimisation (lazy loading of packages, progress indicators)
35. Documentation (user guide, technical docs)
36. Security review (network audit, code review)
37. Reproducible R script export

**Deliverable:** Production-ready v1.

## Phase 5: Additional File Formats (If Phase 0 Passed)

38. Stata (.dta) support via `haven`
39. SPSS (.sav) support via `haven`
40. Excel (.xlsx/.xls) import via `readxl`
41. Mixed models via `lme4` (experimental — labelled as such in UI, disabled if benchmark shows >30 s for simple model)
42. Interactive DAG editing (if `ggdag` / `dagitty` reliable)

**Deliverable:** Extended v1 with full file format support.

---

## MVP SCOPE (v1)

### Included:

- CSV file upload (+ other formats if Phase 0 passes)
- Variable label extraction and editing
- Variable type detection and override

- All basic statistical analyses (descriptive, t-tests, chi-square, ANOVA, linear regression, logistic regression, survival analysis)
- Multi-step wizard UI with progress indicator
- Quick Mode for experienced users
- Table 1 with proper formatting and n column
- Analysis results tables formatted via gt
- Auto-generated plots for each analysis type
- Multiple outcomes support
- Copy-to-clipboard for all tables (pastes into Word with formatting)
- Download: HTML tables, CSV, LaTeX, PNG/SVG plots
- HTML combined report
- Causal reasoning module (static DAG, classification-based)
- Assumptions checking (non-blocking warnings)
- Multiple comparison notes/corrections
- Privacy messaging
- Clustered data handling: notation always available; cluster-robust SEs if sandwich passes Phase 0; mixed model (experimental) if lme4 passes
- Excel export (if writexl passes Phase 0)

**If time pressure requires cutting v1 scope, cut in this order (first to go → last to go):**

1. Static DAG outputs (keep the causal role classification, drop the rendered diagram)
2. Mixed models (experimental anyway; core value is in standard regression)
3. Multiple-outcome combined tables (keep separate per-outcome tables, drop the merged format)
4. Excel export via writexl (CSV is always available)
5. Diagnostic plots (keep assumptions warnings, drop the plots)

Everything below #5 is core and should not be cut.

**Deferred to v2:**

- Word (.docx) report generation (JavaScript-based)
  - Interactive DAG editor (drag-drop nodes and arrows)
  - Drag-and-drop model specification UI
  - Session persistence (save and resume)
  - User accounts / login
  - Analysis history
  - Custom analysis templates
  - Advanced imputation (MICE)
  - Power analysis / sample size calculators
  - PDF report generation
  - Two-way ANOVA / factorial designs
  - Repeated measures ANOVA (if lme4 not available in v1)
  - Additional table styles (AMA, Vancouver, JAMA)
- 

## **EXPLICITLY OUT OF SCOPE (v1)**

These features will not be built, discussed, or half-implemented. This list exists to prevent scope creep.

- Multiple imputation (MICE or any variant)
- Survey weights and complex survey designs (strata + PSU)
- Bayesian models
- Time-varying covariates
- Competing risks survival models
- Meta-analysis
- Propensity score matching/weighting
- Instrumental variables
- Difference-in-differences

- Interrupted time series
- Network analysis
- Factor analysis / structural equation modelling
- Machine learning models (random forests, boosting, etc.)
- Power analysis / sample size calculators
- PDF report generation (requires Cairo/LaTeX in WASM — unlikely)
- Real-time collaboration or shared sessions
- Any server-side processing or data storage

If a user asks for any of these, the app should display: “This analysis is not currently supported. We recommend using R/Stata/SAS directly for this type of analysis.”

---

## STATISTICAL RESPONSIBILITY DISCLAIMER

The following disclaimer should appear in the app footer, the About/FAQ section, and at the top of every downloaded report:

This tool automates common statistical analyses but does not replace statistical judgement. Users are responsible for ensuring that the chosen analysis is appropriate for their data, that model assumptions are met, and that results are interpreted correctly. For complex analyses or high-stakes decisions, consult a statistician.

This is not legalese — it is academic honesty and sets the right expectations.

---

## DELIVERABLES

### 1. Shinylive Application

- Modular Shiny app (module structure as specified above)
- All dependencies declared
- Deployed to GitHub Pages with Shinylive export
- Works on Chrome, Firefox, Safari, Edge

### 2. README.md

- Project description and target audience
- How to run locally: install R + packages → `shiny::runApp()`
- How to deploy: GitHub Pages with `shinylive::export()`
- Security explanation: client-side processing, no data transmission
- Phase 0 results: which packages are confirmed, which fell back
- Testing instructions

### 3. Test Datasets

Synthetic datasets generated via R scripts (reproducible, guaranteed safe):

- `test_csv.csv` — 500 rows, mix of variable types, some missing data
- `test_clustered.csv` — 500 rows with student-in-school clustering
- `test_survival.csv` — time-to-event data with censoring
- `test_binary_outcome.csv` — binary outcome with continuous and categorical predictors
- `test_stata.dta` — labeled variables (if haven available)
- `test_spss.sav` — labeled variables (if haven available)

### 4. Documentation

- User guide with step-by-step screenshots and example workflows
- FAQ section (privacy, supported analyses, troubleshooting)
- Technical documentation (code architecture, function descriptions)
- Phase 0 validation log

### 5. Code Quality

- Comprehensive comments in all modules
- Tidyverse style guide
- Modular functions (<50 lines each)
- Error handling with `tryCatch` and user-friendly messages
- Graceful degradation when optional packages are unavailable

---

## ACCESSIBILITY

- All interactive elements keyboard-navigable
- ARIA labels on custom UI components
- Alt-text for generated plots (auto-generated from analysis description)
- Colour schemes are colorblind-friendly (viridis palette)
- Tables are screen-reader compatible (proper HTML table structure via gt)
- No drag-and-drop required for core functionality (standard select inputs are primary)
- Font sizes meet WCAG AA contrast requirements